

Projet Python — Groupe 2

Complexité, jointures avec pandas et Python idiomatique

Enseignante : Rahma GARGOURI — 2026-2027

Présentation finale — Projet en groupe

Vous préparez une **présentation** (PowerPoint, PDF ou autre format) qui répond aux questions ci-dessous, accompagnée d'un **notebook Jupyter** contenant les solutions des exercices, à exécuter pendant la présentation.

Partie A — Complexité

Questions

1. Expliquez la complexité à quelqu'un qui n'a jamais fait d'informatique.
2. Quel est le coût (la complexité) d'une recherche dans un dictionnaire, et celui d'une recherche dans une liste ? Pourquoi ce n'est pas le même coût ?

Exercices

A.1 — Two Sum. Écrire `deux_sommes(nombres, cible)` qui renvoie les indices de deux nombres, situés à des positions différentes, dont la somme vaut `cible` (ou `None` s'il n'y en a pas). Écrire **deux** versions : une avec deux boucles imbriquées, une avec un dictionnaire. Donner la complexité de chacune.

```
deux_sommes([2, 7, 11, 15], 9)    # (0, 1)
```

A.2 — Three Sum. Écrire `trois_sommes(nombres)` qui renvoie la liste de tous les triplets de nombres, situés à des positions différentes, dont la somme vaut 0. Chaque triplet est trié par ordre croissant, et un même triplet n'apparaît qu'une seule fois. Écrire **deux** versions : une version naïve avec trois boucles imbriquées, puis une version en $O(n^2)$ qui commence par trier la liste. Donner la complexité de chacune.

```
trois_sommes([-1, 0, 1, 2, -1, -4])    # [[-1, -1, 2], [-1, 0, 1]]
```

A.3 — Plus longue sous-chaine sans caractère répété. Écrire `plus_longue_sous_chaine(texte)` qui renvoie la longueur de la plus longue **sous-chaine** (suite de caractères **consécutifs** du texte) qui ne contient aucun caractère répété. Écrire **deux** versions : une version naïve qui teste toutes les sous-chaînes, puis une version en $O(n)$ qui utilise un dictionnaire. Donner la complexité de chacune.

```
plus_longue_sous_chaine("abcabcbb")    # 3 ("abc")
plus_longue_sous_chaine("bbbbbb")      # 1 ("b")
plus_longue_sous_chaine("pwwkew")      # 3 ("wke")
# attention : "pwke" n'est pas une sous-chaine (caractères non consécutifs)
```

Partie B — Jointures avec pandas

Questions

1. Qu'est-ce que `pandas` ?
2. Comment l'installer ?
3. Expliquez les différents types de jointure.
4. Quelle est la différence entre `join`, `merge` et `concat` ?

Exercices

B.1 — Les quatre jointures. Avec les deux tables ci-dessous, réaliser les jointures `inner`, `left`, `right` et `outer` sur la clé `devise`. Avant d'exécuter, **prévoir** le nombre de lignes de chaque résultat, puis vérifier.

```
gauche = pd.DataFrame({"devise": ["USD", "GBP", "JPY"],
                        "pays": ["États-Unis", "Royaume-Uni", "Japon"]})
droite = pd.DataFrame({"devise": ["USD", "JPY", "CHF"],
                        "taux": [1.08, 162.5, 0.94]})
```

B.2 — Lignes sans correspondance. Avec les mêmes tables, trouver les devises de `gauche` absentes de `droite` (résultat attendu : GBP).

Partie C — Python idiomatique

Questions

1. Qu'est-ce qu'une compréhension de liste et une compréhension de dictionnaire ?
2. Quelle est la différence entre `==` et `is` ? Pourquoi écrit-on `x is None` plutôt que `x == None` ?

Exercices

C.1 — FizzBuzz en une ligne. Construire, avec **une seule compréhension de liste**, la liste des nombres de 1 à 15 où les multiples de 3 sont remplacés par "Fizz", ceux de 5 par "Buzz", ceux de 3 et 5 par "FizzBuzz", et les autres par le nombre écrit en texte.

C.2 — Associer deux listes. À partir des deux listes ci-dessous, construire avec `zip` un dictionnaire `date → taux`, puis trouver en une ligne la date du taux le plus élevé.

```
dates = ["2026-01-02", "2026-01-05", "2026-01-06"]
taux = [1.0401, 1.0388, 1.0412]
```

C.3 — `==` contre `is`. Prédire ce qu'affiche chaque `print`, puis exécuter le code pour vérifier, et expliquer chaque résultat avec la fonction `id()`.

```
a = [1, 2, 3]
b = [1, 2, 3]
c = a
print(a == b)
print(a is b)
print(a is c)
c.append(4)
print(a)
print(a == b)
```